COMPUTER VISION COURSE (2024/25)

Master in Electrical and Computers Engineering

Laboratory Assignment 1

Image Filtering and Hough Transform

Introduction

This assignment involves the implementation of a Hough Transform-based line detector using MATLAB. The goal is to develop a pipeline for identifying the start and end points of straight-line segments in images. Key image processing algorithms, including convolution-based filtering, edge detection, and the Hough Transform, were implemented from scratch without relying on pre-built functions from MATLAB's Image Processing Toolbox. The pipeline was tested on a set of provided images, and the impact of various parameters on performance was evaluated. This task provided an opportunity to explore the challenges of parameter tuning and algorithmic adaptation in image processing.

Methodology

The line detection pipeline consists of a series of sequential steps. The process begins with an input image, which is first smoothed using convolution to reduce noise. Edge detection is then applied to highlight boundaries and significant features in the image. The Hough Transform converts these edges into a parameter space, where lines are represented by peaks in the accumulator. The most prominent lines are extracted using non-maximal suppression and sorted by their strength. Finally, line segments are fitted to the detected lines, and the results are overlaid on the original image. This pipeline transforms raw image data into a set of detected lines, providing a clear representation of the image's structural features.

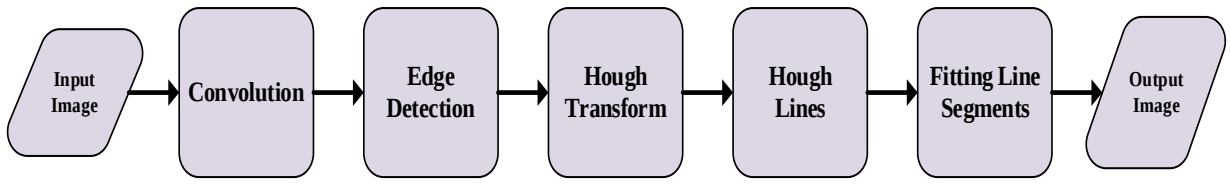


Fig. 1.01: Line Detection Pipeline: Methodology Overview

Experiments:

Convolution

Convolution is a fundamental operation in image processing that applies a filter (or kernel) to an image, modifying its features based on the kernel's properties. The ImageFilter function takes an input image and a kernel as inputs. To handle edge pixels, the image is padded using nearest-neighbor replication, ensuring the convolution operation can be applied uniformly across the entire image. A sliding window then moves across the image, extracting regions of the same size as the kernel. For each position, the kernel is multiplied element-wise with the overlapping region, and the results are summed to produce a single value for the corresponding pixel in the output image. This process is repeated for every pixel, resulting in a filtered image. Convolution is widely used for tasks like smoothing (e.g., Gaussian blur), sharpening, and edge detection. Fig. 1.02 and 1.03 shows the original image and the output of convolution respectively.

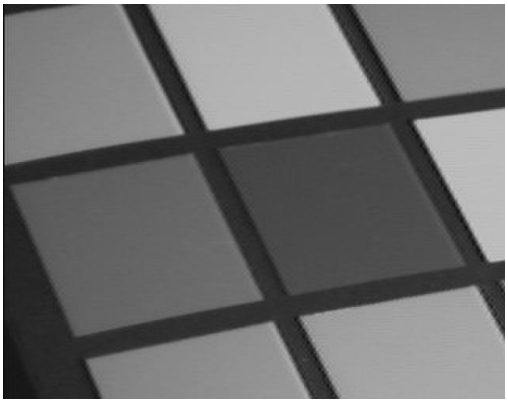


Fig. 1.02: Original Image

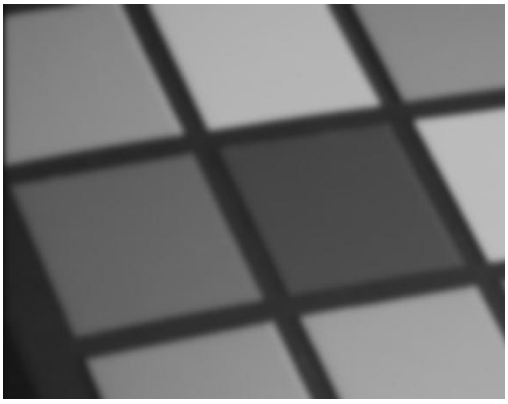


Fig. 1.03: Convolution Result

Edge Detection

Edge filtering identifies boundaries and significant features in an image by detecting regions of rapid intensity change. The EdgeFilter function begins by smoothing the input image using a Gaussian filter, which reduces noise and ensures edge detection is less sensitive to small variations. The filter size is determined by the parameter sigma. Next, gradients in the horizontal and vertical directions are computed using Sobel filters, which highlight areas where pixel intensity changes sharply. The gradient magnitude and direction are then calculated to determine the strength and orientation of edges. Finally, non-maximum suppression is applied to thin the edges, retaining only the most prominent ones. This step ensures that edges are represented as sharp, single-pixel lines. Fig. 1.04 and 1.05 shows the original image and the output of edge filtering respectively.

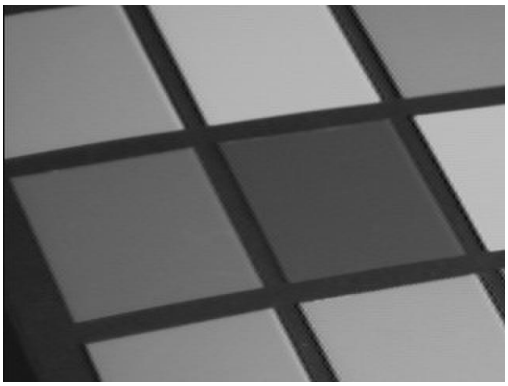


Fig. 1.04: Original Image

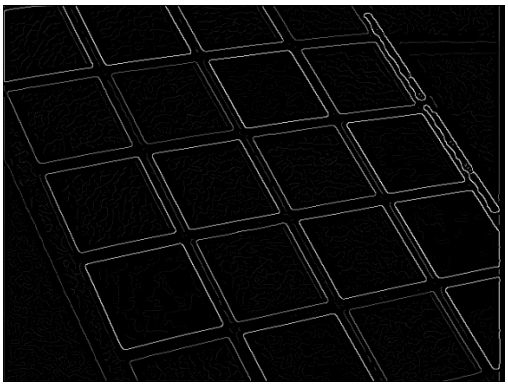


Fig. 1.05: Edge Detection Result

Hough Transform

The Hough Transform detects lines in an image by converting edge pixels into a parameter space defined by rho (distance from the origin) and theta (angle). The HoughTransform function takes an edge-detected image, a threshold to identify edge pixels, and resolutions for rho and theta. It calculates the possible ranges for rho and theta and initializes an accumulator matrix to count votes for potential lines. Each edge pixel votes for lines in Hough space, and the accumulator records these votes. Peaks in the accumulator correspond to detected lines, which can then be extracted and visualized. This method is robust for detecting straight lines even in noisy images. Fig. 1.06 and 1.07 shows the original image and the output of Hough transform outputs respectively.

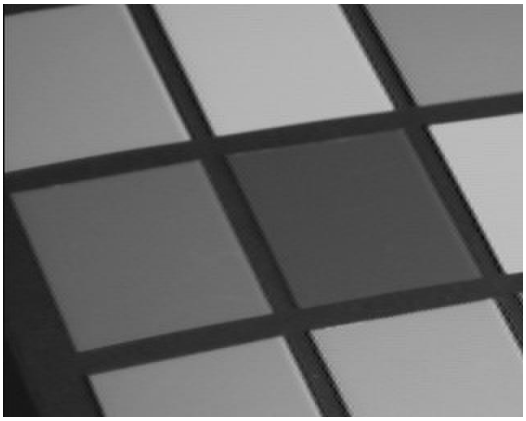


Fig. 1.06: Original Image

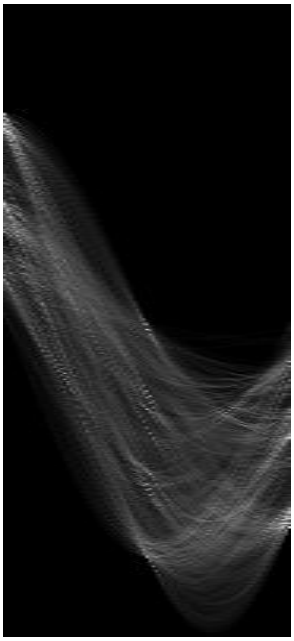


Fig. 1.07: Hough Transform Output

Hough Lines

The HoughLines function extracts the most prominent lines from the Hough accumulator matrix. It begins by applying non-maximal suppression to the accumulator, ensuring that only the strongest local maxima are retained. This is done by comparing each value in the accumulator to its neighbors within a defined window size and suppressing values that are not the maximum in their local region. After suppression, the top nLines peaks are identified by sorting the accumulator values in descending order. These peaks correspond to the most significant lines in the image. The function then converts the indices of these peaks into rho and theta values, which represent the detected lines in Hough space. These parameters can later be used to draw the lines on the original image. Below is an example showing the Hough space and the detected lines overlaid on the original image. Fig. 1.08 and 1.09 shows the original image and output of Hough Lines.

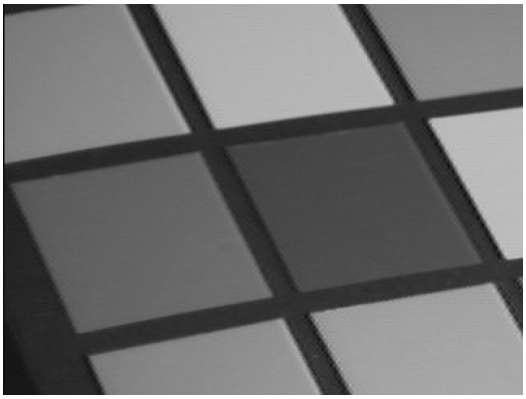


Fig. 1.08: Original Image

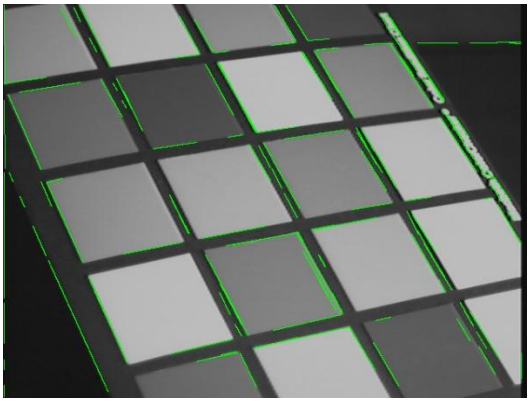
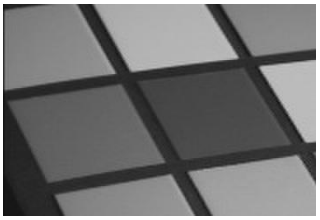


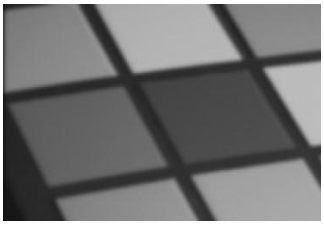
Fig. 1.09: Line Segment Output

Visualizing the Pipeline

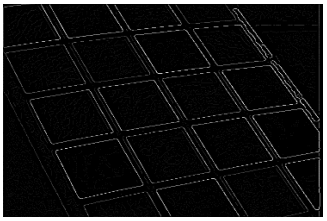
The following images illustrate the output of each step in the line detection pipeline, from the original input image to the final detected line segments.



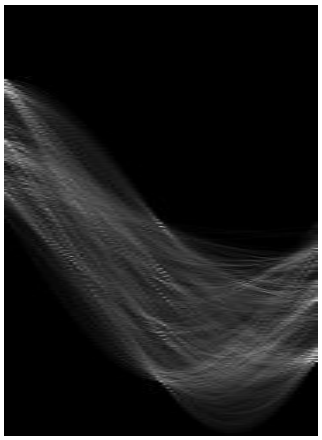
Original Image



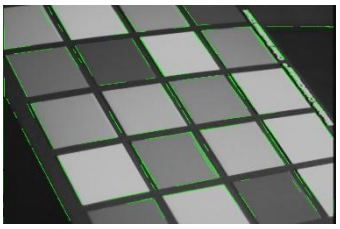
Convolution Output



Edge Detection Result



Hough Transform Output



Line Segment Output

Fig. 1.10: Visualization of the total pipeline (Top to Bottom)

Effect of Parameters on Different Images

The performance of the line detection pipeline depends heavily on the following parameters:

1. **Sigma (σ)**:
    - Controls the smoothing level in the Gaussian filter.
    - **Low σ**: Less smoothing, retains finer details but is more sensitive to noise.
    - **High σ**: More smoothing, reduces noise but may blur edges, making them harder to detect.
    - **Impact**: For images with fine details (e.g., chessboard, Rubik's cube), a lower σ might work better. For noisy images (e.g., road, zebra crossing), a higher σ could improve edge detection.
2. **Edge Detection Threshold**:
    - Determines which gradient magnitudes are considered edges.
    - **Low Threshold**: Detects more edges, including weak ones, which may include noise.
    - **High Threshold**: Detects only strong edges, potentially missing weaker but important lines.
    - **Impact**: For images with clear edges (e.g., building, hall entrance), a higher threshold works well. For images with subtle edges (e.g., packets, cup), a lower threshold might be necessary.
3. **Rho Resolution (rhoRes)**:
    - Controls the granularity of the distance parameter (ρ) in the Hough Transform.

- **Low rhoRes**: Higher precision but requires more computation and may produce redundant lines.
- **High rhoRes**: Lower precision but faster computation, potentially missing closely spaced lines.
- **Impact**: For images with closely spaced lines (e.g., chessboard, zebra crossing), a lower rhoRes is better. For images with widely spaced lines (e.g., road, building), a higher rhoRes may suffice.

4. **Theta Resolution (thetaRes)**:
   - Controls the granularity of the angle parameter (θ) in the Hough Transform.
   - **Low thetaRes**: Higher angular precision but more computation.
   - **High thetaRes**: Lower angular precision but faster computation.
   - **Impact**: For images with lines at varying angles (e.g., Rubik's cube, packets), a lower thetaRes is preferable. For images with mostly horizontal/vertical lines (e.g., building, chessboard), a higher thetaRes may work.
5. **Number of Lines (nLines)**:
   - Determines how many of the strongest lines are extracted.
   - **Low nLines**: Misses weaker but potentially important lines.
   - **High nLines**: Includes more lines, which may include noise or irrelevant lines.
   - **Impact**: For images with many lines (e.g., chessboard, zebra crossing), a higher nLines is needed. For simpler images (e.g., road, cup), a lower nLines may suffice.

Answers to the Questions

1. **Did your code work well on all the images with a single set of parameters?**

   Answer:

   No, the code did not perform equally well on all images with a single set of parameters. Images with clear, well-defined edges (e.g., building, chessboard) produced good results. However, images with noise, subtle edges, or complex patterns (e.g., Rubik's cube, packets) required adjustments to parameters like sigma and edge threshold for optimal performance.

2. **How did the optimal set of parameters vary with images?**

   Answer:

   The optimal parameters varied depending on the image characteristics:

   Sigma: Higher for noisy images (e.g., road, zebra crossing), lower for detailed images (e.g., chessboard, Rubik's cube).

   Edge Threshold: Higher for images with strong edges (e.g., building, hall entrance), lower for images with weak edges (e.g., packets, cup).

   Rho and Theta Resolution: Lower for images with closely spaced or angled lines (e.g., chessboard, packets), higher for simpler images (e.g., road, building).

Number of Lines: Higher for images with many lines (e.g., chessboard, zebra crossing), lower for simpler images (e.g., cup, road).

3. **Which step of the algorithm causes the most problems?**

   Answer:

   Edge Detection caused the most problems. The performance of the entire pipeline depends heavily on the quality of edge detection. If edges are missed or noise is included, the Hough Transform and line detection steps are affected. For example, in images with noise (e.g., road, zebra crossing), edge detection often included irrelevant details, while in images with subtle edges (e.g., packets, cup), important edges were sometimes missed.

4. **Did you find any changes you could make to your code or algorithm that improved performance?**

   Answer:

   Yes, the following improvements could enhance performance:

   Adaptive Thresholding: Use an adaptive edge detection threshold based on image characteristics.

   Dynamic Sigma: Adjust sigma dynamically based on the noise level in the image.

   Post-Processing: Add a step to filter out short or irrelevant lines after detection.

   Parameter Tuning: Allow parameters to be adjusted per image for better results.

5. **Describe how well your code worked on different images, what effect do the parameters have, and any improvements made to your code.**

   Answer:

   The code worked well on images with clear, strong edges (e.g., building, chessboard) but struggled with noisy or complex images (e.g., Rubik's cube, packets). Parameters like sigma, edge threshold, and Hough resolutions significantly impacted the results. For example, a higher sigma improved results for noisy images, while a lower edge threshold helped detect subtle edges. Improvements such as adaptive thresholding and dynamic parameter tuning could make the algorithm more robust across different images.

Summary Table: Parameter Impact on Different Images

Image	Optimal Sigma	Optimal Threshold	Optimal rhoRes	Optimal thetaRes	Optimal nLines	Remarks
img00	Low	Medium	Low	Low	High	Contains overlapping squares; requires high precision for closely spaced lines.
img01	Medium	Low	Medium	Low	High	Rubik's cube; needs lower threshold for subtle edges and high nLines for details.
img02	Medium	Low	Medium	Medium	Medium	Cup and a packet; moderate settings for balanced edge detection.
img03	High	High	High	High	Medium	Hall entrance; strong edges, less noise, simpler lines.
img04	High	High	High	High	Low	Empty road; high sigma and threshold to reduce noise, fewer lines needed.
img05	Medium	Low	Medium	Medium	High	Packets; lower threshold for subtle edges, high nLines for multiple lines.
img06	Medium	Low	Medium	Medium	High	Biscuit packets; similar to img05, requires high nLines for details.

img07	High	High	High	High	Medium	House; strong edges, less noise, moderate nLines.
img08	High	High	High	High	Medium	Rectangle building; similar to img07, strong edges, fewer lines.
img09	Low	Medium	Low	Low	High	Chessboard; requires high precision for closely spaced lines.
img10	High	High	Low	Low	High	Zebra crossing; high sigma for noise reduction, low rhoRes for precision.
img11	High	High	Low	Low	High	Zebra entrance; similar to img10, requires high precision.

Conclusion

This project implemented a Hough Transform-based line detection pipeline in MATLAB, successfully detecting lines in a diverse set of 12 images, ranging from simple structures like buildings and roads to complex patterns like chessboards and Rubik's cubes. While the algorithm performed well on images with clear, strong edges, it faced challenges with noisy or subtle edges, highlighting the importance of parameter tuning for optimal results. Key insights include the critical role of edge detection as a bottleneck, the sensitivity of the pipeline to parameters like sigma and edge threshold, and the need for adaptive tuning to handle varying image characteristics. Future improvements, such as adaptive thresholding and dynamic parameter adjustment, could enhance robustness and accuracy. Overall, the project demonstrated the effectiveness of the Hough Transform for line detection while revealing opportunities for further refinement in handling complex and noisy images.